Actividad: “Manipulación de Cadenas y Punteros”

E.E.S.T N°1 de Vicente López

Curso: Laboratorio de Programación 5°3°

Alumno: Benjamin Hlynczuk

Profesor: Ganduglia Yamil & York Elias Mansilla Muñoz

Introducción

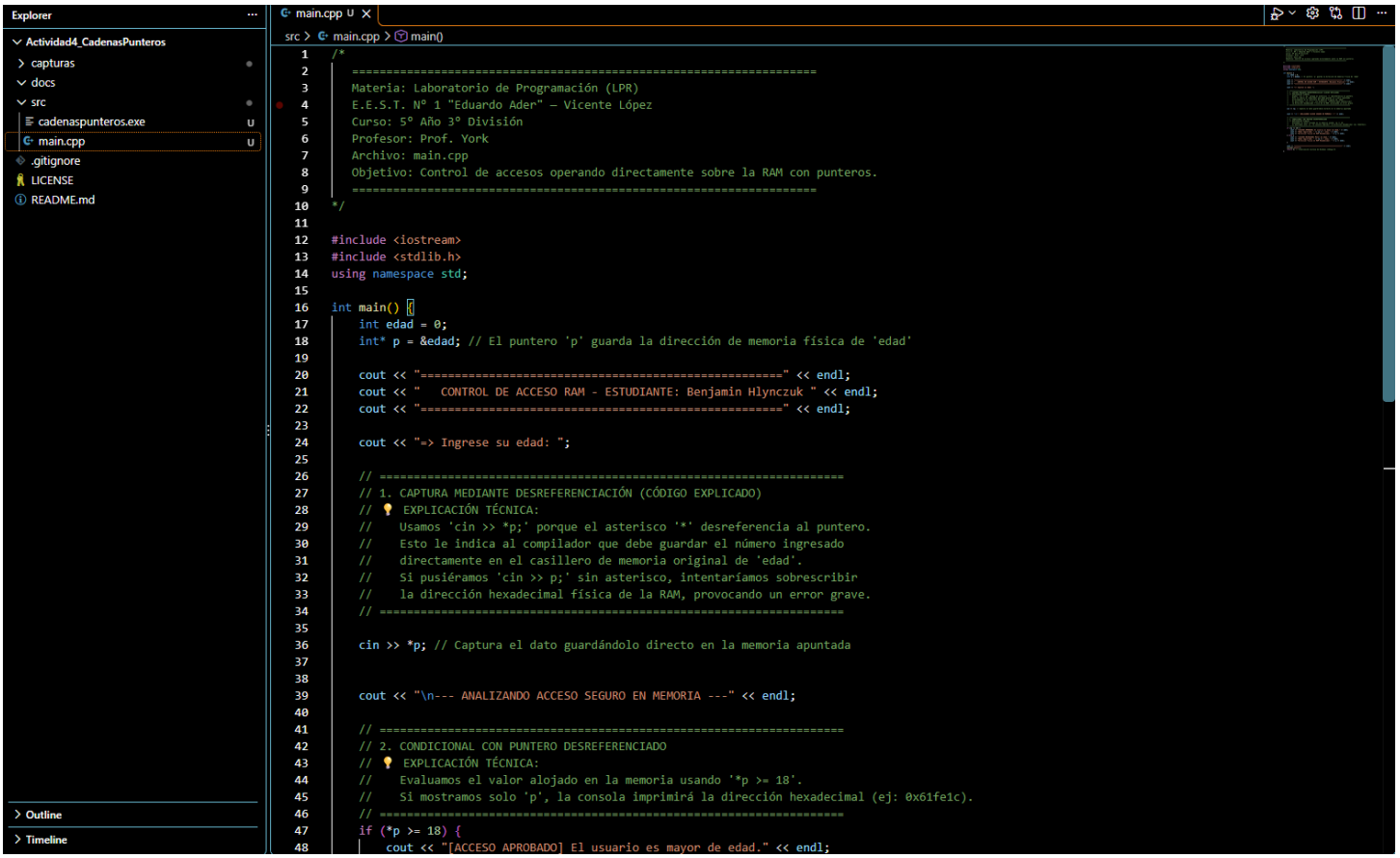
El objetivo de la actividad es comprender cómo se manejan los datos en la memoria RAM mediante el uso de punteros y de la desreferenciación.

Para ello se analicé, compilé y probé un programa de control de acceso por edad. El programa declara una variable entera, guarda su dirección de memoria en un puntero y utiliza el operador de desreferenciación (`*`) para almacenar y evaluar la edad ingresada. Según el valor, muestra si el acceso es aprobado o rechazado, junto con la dirección física en formato hexadecimal.

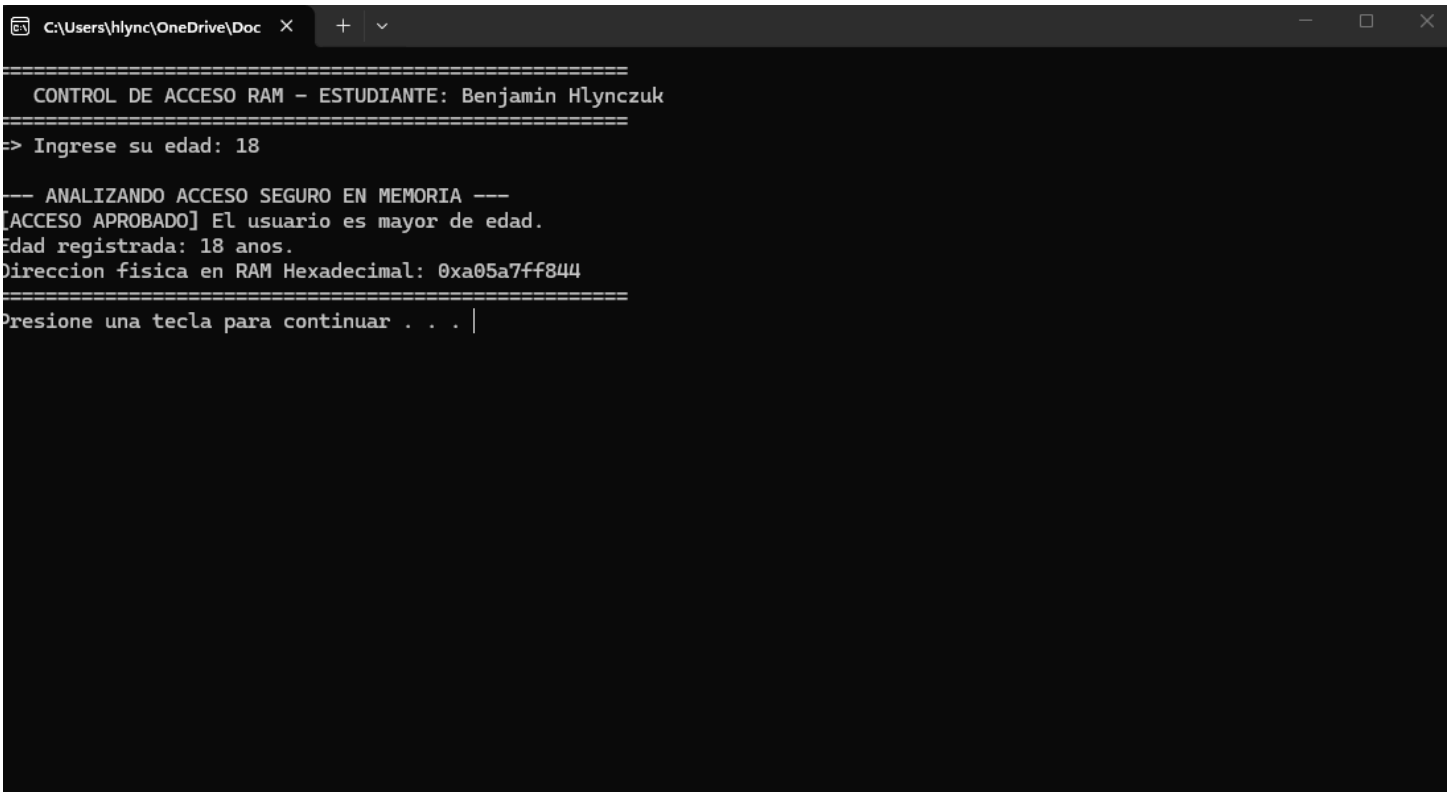
El trabajo se organizó en un repositorio de GitHub con la estructura de carpetas solicitada (`src/`, `docs/` y `capturas/`), e incluye el código fuente, el archivo README, la captura de la ejecución y un esquema de la memoria RAM. Por último, el informe presenta las respuestas al cuestionario técnico de control sobre la diferencia entre `&` y `*`, la desreferenciación al leer datos y la notación hexadecimal.

Desarrollo

Después de crear toda la estructura del repositorio y añadir el código base, cambie el nombre dentro del cout, depure y corrí el código. En la terminal aparecía: la edad, si es mayor de edad o no y la ubicación en memoria donde se guardó el dato de edad.



Captura del codigo fuente



Captura del ejecutable

Cuestionario tecnico de control

Pregunta 1: La instrucción `cout << p;` imprime la dirección de memoria que guarda el puntero, es decir, la ubicación de la variable `edad` en la RAM, expresada en hexadecimal (por ejemplo, `0x61fe1c`). En cambio, `cout << *p;` desreferencia el puntero e imprime el valor contenido en esa dirección, que es la edad ingresada (por ejemplo, `20`). En resumen, `p` muestra *dónde* está el dato y `*p` muestra *qué* dato hay guardado ahí.

Pregunta 2: Es obligatorio desreferenciar porque `p` contiene una dirección de memoria y no el valor de la edad. Con `cin >> *p;`, el dato ingresado se guarda en el casillero al que apunta `p`, que es el de la variable original `edad`. Con `cin >> p;` se estaría intentando escribir sobre el puntero mismo, es decir, sobre la dirección que guarda. Eso no tiene sentido para lo que se busca y, de hecho, el compilador lo rechaza, porque `cin` no sabe leer un dato de tipo `int*`. Aun si se pudiera hacer, `p` dejaría de apuntar a `edad` y el programa podría acceder a posiciones de memoria inválidas, lo que provocaría errores graves.

Pregunta 3: Se utiliza la notación hexadecimal porque es mucho más compacta y legible que el binario, y se relaciona de forma directa con él: cada dígito hexadecimal equivale exactamente a 4 bits, y un byte se representa con 2 dígitos. Las direcciones de memoria son números muy largos que internamente se manejan en binario, y escribirlos en decimal no permitiría ver esa correspondencia, porque la base 10 no es una potencia de 2. El hexadecimal, en cambio, permite leer, comparar y depurar direcciones con facilidad, y por eso se volvió el estándar en programación de bajo nivel. El prefijo `0x` indica que el número está en base 16.

Conclusión

En la actividad, se trabajó sobre temas de menor nivel como la creación, lectura y modificación de datos en memoria RAM. Esto logrado mediante un programa simple que, a partir de la edad, indica estado de adultez.

Referencias:

Repositorio Github: Link

Laboratorio de Programación 5°3°

Actividad: 📝📺📂 ACTIVIDAD 4: MANIPULACIÓN DE CADENAS Y PUNTEROS EN C/C++ - L…